

# Four Architecture Thought Leaders in Detail: Hohpe · Fowler · Martin · Hohmann

Their works, their core concepts, and what they yield for the situational picture, staff work and our software

**Version 1.1** · As of 2026-08-14 · Audience: portfolio and solution leaders and architects · Companion memorandum to the Team-of-Teams memorandum (Part C.4) and the EA memorandum (Part A2)

**About this document:** Curated, conceived and editorially owned by **Robert Seebauer**; researched and produced with the support of **Claude** (Anthropic – model **Claude Fable 5**, via Claude Code). This is **version 1.1**. The short versions of these four profiles live in the [Team-of-Teams memorandum](#) (Part C.4, staff/leadership relevance) and the EA memorandum (Part A2, EA transfer); this companion memorandum is the detailed elaboration. Earlier editions: [release archive](#) [denkschriften/<date>](#) .

## Executive Summary

**Guiding question:** Four practice-oriented thought leaders of software architecture – Gregor Hohpe, Martin Fowler, Robert C. Martin, Luke Hohmann – carry the memoranda series as "civilian corroborating witnesses". What exactly is *in* their works, how do the concepts interlock – and what follows concretely for the situational picture, staff training and the SituationReport software?

### THE SINGLE MOST IMPORTANT SENTENCE IN THIS DOCUMENT

Four paths, one destination: architecture work is **deciding under uncertainty with a shared language** – its tools (patterns, options, boundaries, games) are therefore **leadership and staff tools**, not technology internals.

### Findings in five sentences:

- 1 All four solve the same translation problem** – Hohpe rides the elevator between board and engine room, Fowler and Martin give the levels a shared pattern language, Hohmann brings both to one game table. The Rosetta-Stone problem (EA memorandum, finding 3) is their common denominator.
- 2 Pattern languages are doctrine:** 65 integration patterns (Hohpe/Woolf) and 51 enterprise patterns (Fowler) achieve in civilian life what MDMP and planning language achieve in the military – newly assembled teams are immediately effective because the terms are taught and shared.
- 3 Good architecture keeps options open:** Hohpe's "first derivative"/real-options thinking and Martin's Dependency Rule ("maximise the number of decisions not yet made") are two formulations of the same principle – Clausewitz's handling of friction, cast into software.
- 4 Participation creates legitimacy and a shared picture:** Hohmann's serious games (Buy a Feature, participatory budgeting) are decision rituals in which participants see and carry the trade-offs themselves – shared consciousness and empowered execution in one format.
- 5 Consequence for our software:** Four directly implementable derivations – trend/derivative columns per metric (Hohpe), debt quadrant in the NFR/debt register (Fowler), strangler progress metric for modernisation epics (Fowler), profit-stream field per capability (Hohmann) – all connectable to phases A–C of the [software roadmap](#) (Part E.2).

## Part A – Gregor Hohpe: the translator between the floors

**Person and body of work:** Hohpe has lived the mediator role he writes about, repeatedly: chief architect at Allianz, technical director in the Google Cloud CTO office, enterprise strategist at AWS, Smart Nation Fellow in Singapore. His works build on each other: *Enterprise Integration Patterns* (with Bobby Woolf, 2003) gave integration technology its language; *The Software Architect Elevator* (2020) redefined the architect's role; *Cloud Strategy* (2020) and *Platform Strategy* (2024, with Danieli/Landreau) transfer both to the two biggest structural decisions of the present. Continuously extended at [architectelelevator.com](https://architectelelevator.com).

### A.1 The Software Architect Elevator – the role

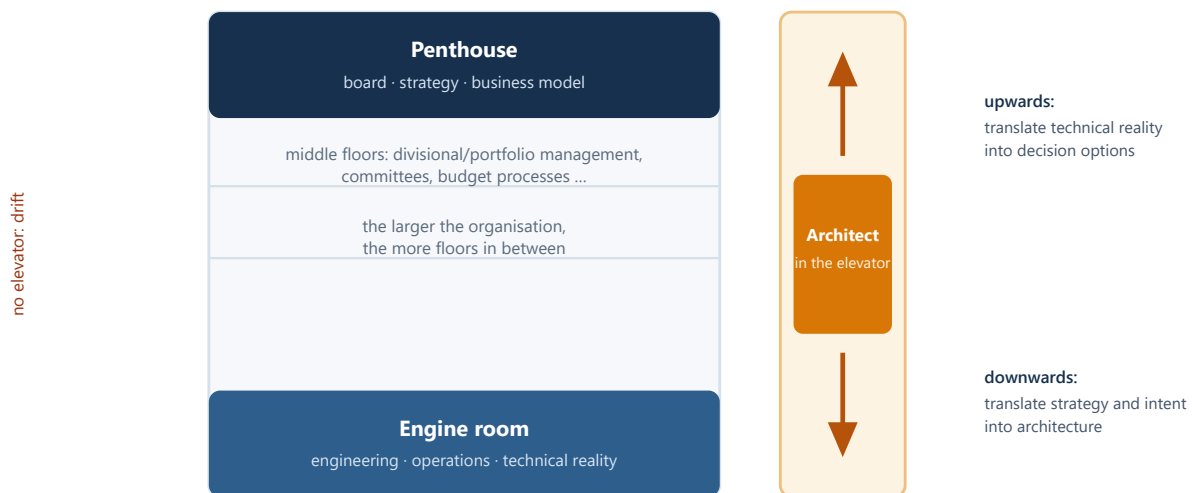


Fig. 1 – The Architect Elevator: value is created by the ride, not by the floor. Without elevator riders, penthouse and engine room drift apart without noticing.

#### A.1.1 The core ideas of the Elevator book

- **The ride is the value.** Architects who only present in the penthouse become slideware architects; those who stay in the engine room leave decisions to people without technical reality. Both together is the role – and it scales with the number of floors (corporate size).
- *"Architects live in the first derivative."* A system's state says little; what matters is **how fast and in which direction** it is changing. Architecture assesses rates of change – and creates structures that increase them (decoupling, automation, feedback).
- **Architecture sells options.** In analogy to financial options: a good architecture decision buys the right (not the obligation) to react more cheaply later – e.g. a scaling option, an exit option, a deployment option. The option's price (abstraction, effort) is weighed against uncertainty: **the more uncertain the future, the more valuable the option.**
- **Organisations are systems.** Hohpe applies systems thinking to the IT organisation itself: without feedback loops (a situational picture!) nobody corrects course; conversely, whoever wants to change structures must change the control loops, not the slides.

## A.2 Enterprise Integration Patterns – the language of integration

65 patterns with name, problem, context and solution – since 2003 *the* reference language for asynchronous integration; messaging platforms from Kafka via RabbitMQ to iPaaS products use this vocabulary verbatim to this day. For the solution level (several ARTs, one solution) integration is the core reality – the most important patterns:

| Pattern (EIP)                                       | What it solves                                                                            | Relevance for the solution level                                                                  |
|-----------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <b>Publish-Subscribe vs. Point-to-Point Channel</b> | The basic decision: one receiver or all interested parties?                               | Degree of decoupling between ARTs – determines who is affected by changes                         |
| <b>Message Translator / Canonical Data Model</b>    | Translating between data models; a shared core model instead of $n \times n$ translations | The data-model counterpart of the capability map: one shared language of business objects         |
| <b>Content-Based Router / Message Filter</b>        | Routing or discarding messages based on content                                           | Makes responsibility cuts visible: who decides where work flows?                                  |
| <b>Splitter / Aggregator / Scatter-Gather</b>       | Decomposing large requests, reassembling partial results                                  | Exactly the pattern of the aggregated situation report: many team contributions → one picture     |
| <b>Correlation Identifier</b>                       | Recognising related messages across systems                                               | End-to-end traceability across ARTs – prerequisite for honest lead-time measurement               |
| <b>Dead Letter Channel / Guaranteed Delivery</b>    | What happens to undeliverable messages?                                                   | The uncomfortable questions of integration reality – belong in the picture as risks               |
| <b>Idempotent Receiver / Competing Consumers</b>    | Making duplicate delivery harmless; distributing load across processors                   | Robustness fundamentals (cf. EA memorandum C.2: CAP, eventual consistency)                        |
| <b>Control Bus</b>                                  | Monitoring and steering the integration landscape itself                                  | EIP's own plea for the situational picture: monitoring as a built-in pattern, not an afterthought |

## A.3 Cloud Strategy & Platform Strategy – structural decisions as strategy

### A.3.1 Cloud Strategy (2020)

Cloud migration is a **decision discipline**, not a technology exercise: the book is built as a collection of decision models (migration paths such as lift-and-shift vs. re-platform vs. re-architect, their true costs and option values), with the core warning that parroted strategy phrases ("cloud first!") do not replace decisions. Transfer: the same discipline – options explicit, criteria upfront, assumptions documented – is what our memoranda series demands for every major portfolio decision (MDMP, module 2).

### A.3.2 Platform Strategy (2024)

**"Innovation through harmonization"**: internal platforms are not a cost-cutting programme; they shift the build/buy line – teams build less undifferentiated plumbing themselves and innovate faster *on* the platform. Success conditions: run the platform as a product (voluntary adoption beats mandates), harmonisation from below instead of standardisation by decree, honest metrics about adoption rather than compliance. Transfer: the platform *is* the architecture runway made operational – its state (adoption, gaps, debt) belongs in the solution picture as the runway dimension.

#### WHAT IT MEANS FOR PICTURE & STAFF WORK

(1) The **vertical liaison** becomes a role in the role mapping (ToT C.1) – staffed with the best, career-relevant. (2) The picture shows **derivatives**: trends, accelerations, tipping points per metric – not just states. (3) Architecture decision papers state the **option value** (which future options does this decision buy or destroy?). (4) Integration reality is reported in **EIP vocabulary** – named patterns instead of prose.

## Part B – Martin Fowler: the evolutionist

**Person and body of work:** Fowler has been Chief Scientist at Thoughtworks since 2000, is a co-signatory of the Agile Manifesto (2001) and runs, with **martinfowler.com** (the "bliki"), probably the most influential living reference work in software development. His books *Refactoring* (1999; 2nd ed. 2018) and *Patterns of Enterprise Application Architecture* (2002) anchored two ideas in the mainstream: software design is **improvable by evolution**, and design knowledge is passed on through **named patterns**.

### B.1 Refactoring – improving structure without changing behaviour

#### B.1.1 The craft

Refactoring is defined as **behaviour-preserving structural improvement in small, safeguarded steps** – a catalogue of named transformations (Extract Function, Move Field, Replace Conditional with Polymorphism ...), each small enough to keep the system running in between. Two consequences: refactoring is **part of daily work**, not a "refactoring project"; and it requires tests – without a safety net, restructuring is gambling. The 2nd edition (2018) modernised the catalogue and the examples.

#### B.1.2 Design-stamina hypothesis & technical-debt quadrant

The **design-stamina hypothesis** (bliki): without investment in internal quality, feature speed drops rapidly as the code base grows – good architecture pays off not "someday" but within weeks. Fowler's answer to "can we afford quality?" is the inversion: "*High quality software is cheaper to produce.*" The **technical-debt quadrant** makes debt steerable instead of moral: deliberate/inadvertent × prudent/reckless – the prudent, deliberate debt ("we ship now and repay, documented") is a legitimate tool; the dangerous one is the inadvertent, reckless kind.

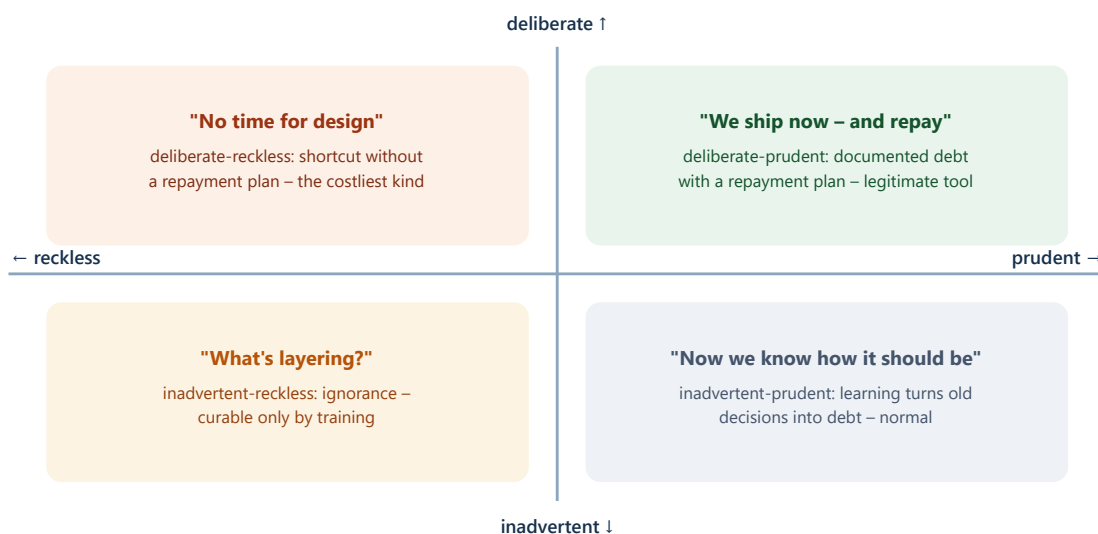


Fig. 2 – Fowler's technical-debt quadrant: debt becomes steerable once its origin is named – classify in the picture instead of moralising.

## B.2 Patterns of Enterprise Application Architecture – the vocabulary

51 patterns for enterprise applications, organised by layers – to this day the reference language in which frameworks name their concepts. The most important for solution discussions:

| Pattern (PoEAA)                                | Core idea and today's relevance                                                                                                              |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Domain Model vs. Transaction Script</b>     | The fundamental decision: rich business logic as an object model or procedural flows? Determines maintainability as complexity grows         |
| <b>Service Layer</b>                           | A defined application boundary with clear operations – the precursor of today's API-first cuts                                               |
| <b>Repository / Data Mapper</b>                | Decoupling the domain model from persistence – the basis of every testable, replaceable data access layer (every ORM speaks this vocabulary) |
| <b>Unit of Work / Identity Map / Lazy Load</b> | Consistent change tracking and loading behaviour – the terms behind every ORM behaviour that feels "magical" to teams                        |
| <b>Gateway / Remote Facade / DTO</b>           | Encapsulating third-party systems, cutting network boundaries coarse-grained – the craft tools of system boundaries                          |
| <b>Optimistic/Pessimistic Offline Lock</b>     | Concurrency across user sessions – the classic behind "lost updates"                                                                         |

## B.3 Strangler Fig & the key bliki articles

### B.3.1 Strangler Fig Application (2004)

Named after the strangler fig that gradually grows around its host tree: a new system grows **piece by piece around the legacy system**, taking over function after function (redirected via facade/routing) until the legacy can die off. The counter-design to the big-bang rewrite – always shippable, risk per step small, progress measurable (share of redirected functionality). Today the standard pattern of every legacy modernisation.

### B.3.2 Further key articles (selection for this briefing)

- **MonolithFirst / MicroservicePrerequisites:** distributed architecture is not a starting point but has to be earned (operational maturity, monitoring, fast deployments) – a warning against architecture fashion.
- **BranchByAbstraction & FeatureToggles:** large restructurings on the living system without long-lived branches – the techniques behind continuous delivery.
- **YAGNI** ("You Aren't Gonna Need It"): speculative generality is borrowing without value in return – the corrective to options thinking: buying options yes, building palaces on suspicion no.
- **Is High Quality Software Worth the Cost?** (2019): the economic short version of the design-stamina thesis – required reading for budget discussions.

#### WHAT IT MEANS FOR PICTURE & STAFF WORK

(1) **Debt quadrant as reporting format:** the NFR/debt register classifies each item by origin – this detoxifies the discussion and makes repayment plannable. (2) **Strangler progress as roadmap metric:** modernisation epics report "share of redirected functionality" instead of "% done". (3) **Pattern vocabulary as doctrine:** the civilian proof of ToT success factor 4 – shared language makes newly assembled staffs immediately effective.

## Part C – Robert C. Martin: boundaries, rules, craftsmanship

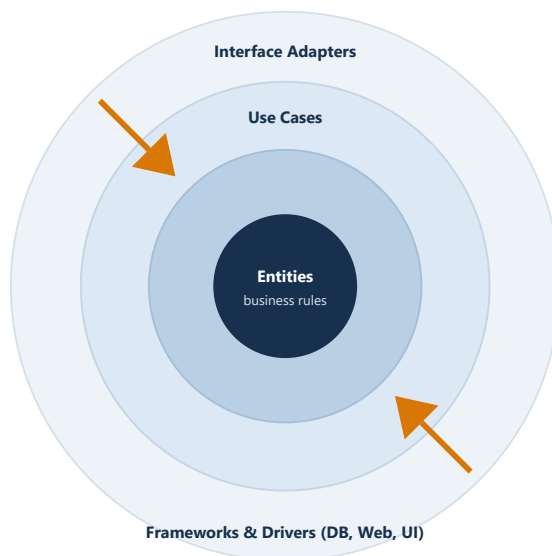
**Person and body of work:** "Uncle Bob" Martin, a developer since the 1970s, co-signatory of the Agile Manifesto and the defining voice of the **software craftsmanship movement**. He bundled and popularised the SOLID principles and wrote two of the industry's most-read books with *Clean Code* (2008) and *Clean Architecture* (2017) – deliberately normative in style, which is both their strength and their weakness (C.4).

### C.1 Clean Code – readability as economics

#### C.1.1 The core ideas

- **Code is read ~10× more often than it is written.** Readability is therefore not aesthetics but the biggest cost lever: expressive names, small single-purpose functions, no surprises.
- **Boy-scout rule:** always leave the code a little cleaner than you found it – quality as continuous micro-investment instead of a big project (congruent with Fowler's everyday refactoring).
- **Tests as first-class citizens:** test code deserves the same care as production code; disciplined development (up to TDD) is a matter of practice, not talent.

### C.2 Clean Architecture – the Dependency Rule



#### The Dependency Rule:

source-code dependencies point exclusively inwards – business rules know nothing of DB, web, UI.

#### Consequences:

- frameworks/DB/UI are *details* – replaceable, testable, deferrable
- good architecture **maximises the number of decisions not yet made** (= options kept open)
- "Screaming Architecture": structure screams the purpose, not the framework

#### SOLID (short form):

Single Responsibility · Open/Closed · Liskov Substitution · Interface Segregation · Dependency Inversion

Fig. 3 – Clean Architecture: concentric rings, dependencies only inwards. Boundaries are the most expensive and most valuable decisions – the same logic applies to solution and ART cuts.

#### C.2.1 What transfers to the solution level

- **Boundaries are investment decisions:** where a boundary runs (between services, ARTs, suppliers) determines what is cheap or expensive to change later – the Dependency Rule is the test criterion: do the dependencies point to the business or to the technology?
- **Defer the "details":** database, framework and UI commitments as late as responsibly possible – literally Hohpe's option selling, phrased from the code perspective.
- **Screaming Architecture as a review question:** can you tell a solution's business purpose from its cut – or only its technology history?

### C.3 Craftsmanship: katas, dojos, discipline

The craftsmanship movement (around Martin, Dave Thomas and others) **institutionalised deliberate practice: code katas** – small, repeatedly trained exercises – and **coding dojos** – group exercises with feedback – turn principles into reflexes. Martin's guiding sentence "*The only way to go fast is to go well*" is the short form: discipline is a prerequisite for speed. That is exactly what general-staff training has taught for 200 years – planning exercises until the form sits. The transfer into the curriculum (ToT Part D) are **planning katas**:

| Planning kata                  | Duration   | Trained skill                                                                               |
|--------------------------------|------------|---------------------------------------------------------------------------------------------|
| <b>Situation-analysis kata</b> | 30 minutes | Extract problem, constraints, means and assumptions from a raw report (MDMP step 2)         |
| <b>Premortem kata</b>          | 60 minutes | Facilitate "it is 12 months later and the undertaking has failed – why?"                    |
| <b>Options kata</b>            | 45 minutes | Develop two real alternatives for an initiative including comparison criteria               |
| <b>BLUF kata</b>               | 15 minutes | Present a situation in 5 sentences: bottom line up front, then options, then recommendation |
| <b>Backbrief kata</b>          | 20 minutes | Explain a received mission back in your own words including its intent                      |

### C.4 Critical appraisal

Martin's tone is deliberately apodictic – "clean" implies that deviation is "dirty". This gave the industry simple, teachable rules (his strength as doctrine) but invites dogmatism: rules without context weighing produce cargo cult (e.g. forcing micro-functions or layers where none are needed). The productive reading of this series: **use as doctrine, read with Carducci's "tailor-made" corrective** (EA memorandum A.2) – master the shared rules first, then deviate with reasons. This is exactly how militaries treat their doctrine: mission command allows deviation, but only after mastering the form.

#### WHAT IT MEANS FOR PICTURE & STAFF WORK

- (1) **Boundary reviews**: regularly test solution/ART cuts against the Dependency Rule – dependency direction is measurable (dependency heatmap).
- (2) **Quality as speed** economically justifies the NFR dimension of the picture.
- (3) **Planning katas** between the curriculum modules turn practice into routine instead of an event.

## Part D – Luke Hohmann: the bridge builder to the business model

**Person and body of work:** Hohmann combines three careers: software engineer and architecture author (*Beyond Software Architecture*, 2003, published in Fowler's Signature Series), entrepreneur (founder of **Conteneo**, whose participatory-budgeting platform scaled portfolio decisions in large enterprises – company built up and sold) and method author (*Innovation Games*, 2006; *Software Profit Streams™*, 2023, with Jason Tanner and Federico González at Applied Frameworks). He contributed to the SAFe framework – its **participatory budgeting** in Lean Portfolio Management goes back to his methodology.

### D.1 Beyond Software Architecture – tarchitecture meets marketecture

#### D.1.1 The two architectures

Hohmann's term pair: the **"tarchitecture"** (technical architecture: components, interfaces, technologies) and the **"marketecture"** (business architecture: licence model, price structure, target segments, deployment, upgrade and support model, branding) describe *the same product* – and fail when designed separately. The book covers the neglected half: licensing and licence enforcement, installation and upgrade as architecture topics (whoever cannot upgrade will not buy again), portability as a business decision instead of a technology reflex, brand and naming questions. Core sentence in essence: **the best technical architecture is worthless if the business architecture does not carry – and vice versa.**

### D.2 Innovation Games – serious games as decision rituals

Twelve "serious games" for product and portfolio decisions, each with a clear question, procedure and evaluation. The most important for portfolio staff work:

| Game                          | The question it answers                                                                                                  | Use in the portfolio context                                                               |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Buy a Feature</b>          | What is it <i>really</i> worth to stakeholders? (limited play budget, joint buying; expensive features force coalitions) | Feature/epic prioritisation with real stakeholders; exercise format in curriculum module 2 |
| <b>Prune the Product Tree</b> | Where should the product grow – and where not? (tree metaphor: branches = capabilities)                                  | Roadmap conversations; the civilian relative of the capability map                         |
| <b>Speed Boat</b>             | What slows us down? (name anchors on the boat instead of criticising people)                                             | Impediment/debt collection without blame – input for the situational picture               |
| <b>Remember the Future</b>    | How will we have recognised success? (narrate backwards from the future)                                                 | Target/intent formulation; related to the premortem (turned positive)                      |
| <b>Product Box</b>            | What does the product promise – in customer words? (have customers design the packaging)                                 | Sharpen the value promise per capability – input for the business reading                  |
| <b>20/20 Vision</b>           | What is more important than what? (pairwise ranking as at the optician)                                                  | Fast priority ranking in committees before WSJF numbers are computed                       |



### D.2.1 Participatory budgeting – the scaled ritual

The portfolio scaling of Buy a Feature: many participants receive **shares of a real budget** and distribute it jointly across competing initiatives – in facilitated rounds in which coalitions, trade-offs and reasons become *public*. The result is not just a ranking but **shared understanding and carried legitimacy**: whoever helped distribute carries the decision. SAFe adopted the method as standard practice for lean budgets (recommended on a six-month/PI rhythm). For our series, participatory budgeting is *the* example of a ritual that creates shared consciousness and empowered execution at the same time – McChrystal's coupling, operationalised as a budget process.

## D.3 Software Profit Streams™ – profitability is designed

### D.3.1 The core ideas (with Jason Tanner, 2023)

- **Profitability is a design discipline:** sustainable profit does not arise as a by-product of good technology but is designed along the lifecycle – with the **Profit Stream Canvas** as the working format.
- **Three pillars:** (a) quantify *value for the customer* (customer ROI – without provable customer benefit no sustainable price), (b) design *value capture* (pricing model, price points, packaging, licence model including compliance/enforcement), (c) *sustainability across the lifecycle* (upgrades, maintenance, ecosystem – the bridge back to Beyond Software Architecture).
- **Licence and pricing decisions are architecture decisions:** metering, multi-tenancy, deployment model and compliance evidence must be carried technically – tarchitecture and marketecture, thought through to the end twenty years later.

#### WHAT IT MEANS FOR PICTURE & STAFF WORK

(1) The **business reading** of the two-readings layout gains substance: profit and cost streams per capability belong in the picture over time – otherwise it shows only the cost half of the truth. (2) **Decision rituals are purchasable:** Buy a Feature and participatory budgeting are available as programs (ToT D.1) – the fastest way to move prioritisation from hallway chatter into a ritual. (3) **Speed Boat and Remember the Future** provide facilitated input formats for risks/debt and intent – lightweight sources for the phase-B artefacts of the roadmap.

## Part E – Synthesis: four paths, one destination

### E.1 Where the four converge

| Thought leader | Core contribution (one line)                                                              | Supports in the EA memorandum ...                                         | Supports in the ToT memorandum ...                                                |
|----------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Hohpe</b>   | Translation as a role; options and rates of change as steering variables                  | Finding 3 (Rosetta Stone), "two readings" principle; runway (platform)    | Role mapping C.1 (vertical liaison); competency field 1 (trends into the picture) |
| <b>Fowler</b>  | Evolution instead of master plan; patterns as shared vocabulary                           | Principle 2 (debt is a first-class citizen); B.2 (modernisation roadmaps) | Success factor 4 (process standard); doctrine thesis B.3.6                        |
| <b>Martin</b>  | Boundaries and dependency direction; discipline as speed                                  | Finding 4 (competence), principle 2 (NFR/quality)                         | Didactics thesis (exercise-based); curriculum katas                               |
| <b>Hohmann</b> | Designing business model and technology together; decision as a ritual with participation | Findings 3 and 5 (business reading, software consequence)                 | Success factors 1+2 (coupling, rituals before tools); B.3 planning games          |

The common line in four guiding sentences: **translate permanently** (Hohpe) · **improve by evolution and name what you do** (Fowler) · **keep boundaries clean and practise the form** (Martin) · **decide together and design the business model too** (Hohmann). Together they form the civilian foundation of what the series derived from the military: picture + doctrine + practice + decentralised, legitimised decision-making.

### E.2 Derivations for our software (SituationReport)

- 1 Trend/derivative columns per metric (Hohpe – "first derivative")**  
Every pooled metric receives direction and acceleration ( $\Delta$  vs. previous PI, trend arrow, tipping-point threshold) – computable from existing Jira data: **phase A** of the roadmap.
- 2 Debt quadrant in the NFR/debt register (Fowler)**  
The phase-B artefact "NFR/runway register" gets, per entry, the classification deliberate/inadvertent × prudent/reckless plus a repayment note – debt is reported steerably instead of debated morally.
- 3 Strangler progress metric for modernisation epics (Fowler)**  
Modernisation epics report "share of redirected functionality/traffic" as the progress measure (instead of "% done") – an honest, measurable indicator; lightweight input: **phase B**.
- 4 Dependency-direction check in the dependency heatmap (Martin – Dependency Rule)**  
The planned dependency heatmap marks dependencies that point "outwards" (business logic depending on technology/suppliers) – boundary violations become visible before they get expensive: **phase B/C**.
- 5 Profit-stream field per capability (Hohmann)**  
The capability map (phase-B artefact) optionally receives value/cost-stream fields per capability – the business reading gets its own data instead of reworded flow metrics.
- 6 Ritual results as decision-log import (Hohmann – Buy a Feature / participatory budgeting)**  
Results of prioritisation sessions (ranking, budget distribution, reasons) flow structured into the decision/assumption log – the picture documents not only *what* was decided but *how it was legitimised*: **phase B**.

## E.3 Reading paths by role

- **Portfolio/solution manager:** Hohpe *Elevator* → Hohmann *Beyond Software Architecture* (marketecture part) → Fowler bliki "Is High Quality Software Worth the Cost?" → Hohmann/Tanner *Software Profit Streams*.
- **Lead/solution architect:** Martin *Clean Architecture* → Hohpe/Woolf *EIP* (catalogue part as reference) → Fowler *PoEAA* + *Refactoring* → Hohpe *Platform Strategy*.
- **STE / PMO / staff roles:** Hohmann *Innovation Games* → Hohpe *Elevator* (communication chapters) → Fowler bliki "StranglerFigApplication" → this memorandum, Part E, as the map.

## Connectivity to the series (memoranda)

1

### Team-of-Teams memorandum

Part C.4 is the short version of this memorandum from the staff/leadership perspective; the planning katas (here C.3) and the Innovation Games program line (there D.1) interlock both documents.

2

### EA memorandum

Part A2 contains the EA-oriented short profiles; the five situational-picture principles from there are substantiated here per author and broken down into software features in Part E.

3

### Reading list

There – in the curator's working archive – the short appraisals of all eight works with LS-focus markers; this memorandum is their detailed elaboration – the software roadmap remains in [docs/portfolio\\_roadmap\\_solution-lagebild.md](#).

## Bibliography

**Gregor Hohpe:** Hohpe/Woolf: *Enterprise Integration Patterns – Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003. · *The Software Architect Elevator – Redefining the Architect's Role in the Digital Enterprise*. O'Reilly, 2020. · *Cloud Strategy – A Decision-Based Approach to Successful Cloud Migration*. Architect Elevator Series, 2020. · Hohpe/Danieli/Landreau: *Platform Strategy – Innovation Through Harmonization*. Architect Elevator Series, 2024. · Web: [architectelevator.com](http://architectelevator.com).

**Martin Fowler:** *Refactoring – Improving the Design of Existing Code*, 2nd ed. Addison-Wesley, 2018 (first edition 1999). · *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. · bliki articles ([martinfowler.com](http://martinfowler.com)): "StranglerFigApplication" (2004), "TechnicalDebtQuadrant" (2009), "DesignStaminaHypothesis", "MonolithFirst", "MicroservicePrerequisites", "BranchByAbstraction", "Yagni", "Is High Quality Software Worth the Cost?" (2019).

**Robert C. Martin:** *Clean Code – A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008. · *Clean Architecture – A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

**Luke Hohmann:** *Beyond Software Architecture – Creating and Sustaining Winning Solutions*. Addison-Wesley, 2003. · *Innovation Games – Creating Breakthrough Products Through Collaborative Play*. Addison-Wesley, 2006. · Tanner/Hohmann (with Federico González): *Software Profit Streams™ – A Guide to Designing a Sustainably Profitable Business*. Applied Frameworks, 2023. · SAFe guidance "Participatory Budgeting" (based on Hohmann's/Conteneo's methodology).

**Internal references:** Team-of-Teams memorandum "Team of Teams & Staff Training" (Part C.4) · EA memorandum "Enterprise Architecture & the Situational Picture at the Solution Level" (Part A2) · situational-awareness reading list in the curator's working archive · software roadmap: [docs/portfolio\\_roadmap\\_solution-lagebild.md](#) (phases A–C).

## Version history

| Version | Date       | Changes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | 2026-08-07 | First edition of the companion memorandum: four detail profiles (Hohpe: Elevator/EIP pattern selection/Cloud & Platform Strategy; Fowler: Refactoring/debt quadrant/PoEAA selection/Strangler Fig & bliki; Robert C. Martin: Clean Code/Dependency Rule/SOLID/katas & critical appraisal; Hohmann: tarchitecture-marketecture/Innovation Games catalogue/participatory budgeting/Profit Streams); synthesis with convergence table, six software derivations (roadmap phases A–C) and reading paths; three figures; DE + EN. |
| 1.1     | 2026-08-14 | Reference maintenance for the online publication: cross-references to the series now link to the online overview, workshop paths removed; cross-references no longer carry version numbers. Content unchanged.                                                                                                                                                                                                                                                                                                               |

Source briefing "Four Architecture Thought Leaders in Detail" (companion memorandum) · version 1.1 · Solution/Portfolio Management · as of 2026-08-14 · curated by Robert Seebauer, produced with Claude Fable 5 (Anthropic) · Current editions online: <https://jaegerfeld.github.io/situation-report/denkschriften/>